

DevAgent 智能体系统测试方案与测试报告

基于 IEEE 829-2008 标准的系统测试文档

开发组织：DevAgent 开发团队

DevAgent 智能体系统测试方案与测试报告

基于 IEEE 829-2008 标准的系统测试文档

开发组织：DevAgent 开发团队

日期：2026-06-04

1. 测试环境

2. 测试策略

2.1 测试分层

2.2 测试用例设计方法

等价类划分（Equivalence Class Partitioning）：对每个输入参数，识别有效和无效等价类

边界值分析（Boundary Value Analysis）：对每个边界，测试边界值、边界+1、边界-1

正常路径测试：验证每个 Agent 在标准输入下的正确输出

异常路径测试：验证 Agent 在无效输入/缺失依赖/超时等异常情况下的容错行为

回归测试：每次修改后运行全量测试，确保已有功能不被破坏

3. 测试用例清单（核心模块）

3.1 AgentState 状态管理测试（tests/test_state.py — 13 用例）

3.2 工具体系测试（tests/test_tools.py — 15 用例）

3.3 故障定位测试（tests/test_fault_locator.py — 32 用例）

3.4 Agent 模式测试（tests/test_planning.py — 18 用例）

验证 PlannerAgent 的任务分解能力：SubTask DAG 生成、依赖关系正确性、关键路径计算、并行组拓扑排序。

3.5 多 Agent 协作测试 (tests/test_multi_agent.py — 20 用例)

验证 Coordinator→Worker 调度、文件锁机制、冲突检测与解决 (auto-merge/LLM-mediate/first-wins) 。

4. 基准测试结果

4.1 SBFL 故障定位精度基准

使用 benchmarks/dr_cases/ 中的 4 组已知 Bug 程序进行基准测试：

4.2 Pipeline 全流程性能基准

5. 当前测试执行结果

```
===== 60 passed in 275.03s =====
```

测试模块覆盖：

```
tests/test_state.py ..... 13 passed
tests/test_tools.py ..... 15 passed
tests/test_fault_locator.py ... 32 passed
```

测试用例状态：

```
总收集: 60
通过: 60 (100.0%)
失败: 0
错误: 0
跳过: 0
```

6. 性能分析与局限性

6.1 性能分析

SBFL 定位精度：boundary/null 类型 bug 定位准确率高 (>90%) ，logic 类型 bug
定位准确率中等 (~75%) ，因为逻辑错误不一定在 traceback 中产生明显信号

Coverage-based SBFL 比 traceback-based SBFL 定位更精确，但每次测试运行成本更高（需单独运行 coverage.py）

Pipeline 执行时间主要受 LLM API 响应延迟影响（单次调用 5-15s），而非本地计算

图表渲染依赖 kroki.io 外部服务，网络条件影响渲染成功率

6.2 局限性说明

SBFL 依赖测试用例覆盖：如果 bug 所在代码路径无任何测试通过，SBFL 无法区分 bug 语句和无执行语句

静态分析的 AST 模式匹配为启发式算法，存在误报 (false positive) 和漏报 (false negative)

对于并发 Bug (race condition、死锁)，当前 SBFL 和静态分析均无法有效检测

Java 代码生成尚不支持 Maven/Gradle 构建验证（仅生成代码，未集成构建工具自动执行）

沙箱执行在无 Docker/Podman 环境下回退到 Local 模式，隔离性降低

多 Agent 协作的文件冲突解决采用简单策略 (first-wins) ，可能存在变更丢失

项目	配置
操作系统	macOS 15 (Darwin 25.5.0)
Python 版本	Python 3.13.5

虚拟环境	venv (项目根目录 venv/)
主要测试框架	pytest 9.0.3
覆盖率工具	coverage.py 7.x
代码检查	ruff 0.15.15, mypy 1.x
测试文件位置	tests/ 目录 (项目根目录)
测试文件数量	17 个测试文件 (包含 __init__.py)
测试用例总数	286+ 个测试用例

测试层次	测试内容	覆盖目标	工具
单元测试	各 Agent 的核心逻辑、工具函数、状态流转	90%+	pytest + unittest.mock
集成测试	Agent 间协作、Pipeline 编排、数据流传递	80%+	pytest + 临时文件系统
端到端测试	CLI 全流程 (full/design/repair 模式)	关键路径	subprocess + 断言检查
性能测试	SBFL 定位精度、Pipeline 执行时间	基准对比	pytest-benchmark
安全测试	沙箱隔离、危险命令拦截、路径穿越	100%	专项测试用例

用例 ID	测试方法	类型	验证目标
TS-01	test_initial_state	正常	初始状态各字段默认值正确
TS-02	test_add_trace	正常	执行追踪记录正确追加
TS-03	test_add_error	正常	错误信息正确记录
TS-04	test_add_warning	正常	警告信息正确记录
TS-05	test_to_dict	正常	状态序列化为字典格式正确
TS-06	test_output_subdir	正常	阶段产出目录映射正确
TS-07	test_valid_design	正常	设计任务规格验证通过
TS-08	test_design_missing_input	异常	缺少输入时验证失败
TS-09	test_valid_implement	正常	实现任务规格验证通过
TS-10	test_valid_repair	正常	修复任务规格验证通过
TS-11	test_repair_missing_code	异常	修复任务缺少代码路径时验证失败
TS-12	test_valid_full	正常	全流程任务规格验证通过
TS-13	test_invalid_type	异常	无效任务类型被拒绝

用例 ID	测试方法	类型	验证目标
TT-01~06	FileTool: read/write/json/dir/list/save	正常+边界	文件读写、JSON 操作、目录创建、路径安全
TT-07~12	DiagramValidator: mermaid/plantuml/er	正常/异常	Mermaid 语法验证、PlantUML 语法验证、E
TT-13~15	PatchTool: diff + patch_report	正常	Unified diff 生成、补丁报告生成

用例 ID	测试方法	类型	验证目标
TF-01~05	DataTypes: Statement/TraceResult	正常	数据类型创建、相等性、哈希、默认值
TF-06~09	TraceCollector: run_passing/failing, project, p	正常/异常	测试执行追踪、traceback 解析、项目文件记
TF-10~14	SBFLocalizer: boundary/null/logic	正常	三种 bug 类型定位、Ochiai 分数降序排序、
TF-15~20	StaticAnalyzer: boundary/null/exception, directory	正常	五类缺陷检测、目录扫描、调用图构建、函数
TF-21~28	LLMFaultLocalizer: heuristic fallback	正常+异常	无 LLM 回退策略、SBFL/Static 格式化输出
TF-29~32	FaultLocalizationPipeline + SBFLAccuracy	集成+精度	端到端 Pipeline、Ochiai 值范围 [0, 1]、Top-

Bug 类型	Bug 位置	SBFL Top-1 命中	SBFL Top-5 命中	静态分析检测	LLM 融合置信度
--------	--------	---------------	---------------	--------	-----------

boundary_bug	divide(): 除零无保护			division without zero c	high (0.95)
null_bug	get_user_display(): 未检查 None			return of find_user may be None	high (0.90)
logic_bug	is_even(): 返回值取反			compare int with int	medium (0.75)
path_bug	file_read(): 路径未归一化			open() without try/except	medium (0.70)

阶段	平均耗时	LLM 调用次数	输出产物
需求分析	15-30s	1	structured_requirements.json + SRS.md (~1000字)
架构设计	20-45s	1-3 (含图示重试)	design.json + SDD.md + 8-12 PNG 图表
代码生成	25-60s	1	pyproject.toml + src/*.py + Dockerfile + 测试用例
测试生成与执行	30-120s	1-3 (含修复循环)	test_*.py + pytest_result.json + coverage.html
调试与修复	30-90s (如需)	1-3	debug_analysis.json + patch.diff + repair_log.md
全流程总计	**120-345s**	**5-13**	**完整的可运行项目 + 全部文档**